



FarAI

法本信息 · 人智协同研发平台

FarAI 研发流水线

让"个人用 AI 变快",真正变成"组织级系统提速"

驾驭框架先行

四闭环工艺

先建安全网

再放大自主度

六章构成"从悖论到落地"的完整叙事

命题 → 平台 → 组织 → 场景 → 工程底座 → 价值实施

01	02	03	04	05	06
核心命题	产品全景	组织重塑	四大场景	工程底座	价值实施
个人提效#组织提效	FarAI 研发流水线	新生产关系	每场景一个闭环	Harness 驾驭框架	试点到规模化
· 核心悖论 · 根因剖析 · FarAI 回答	· 四层架构 · 场景映射	· 新角色 Builder · FDE 三层协同 · 三层流动机制	· 需求/交付闭环 · 知识底座 · 治理 FinOps	· 9 大组件 · 先行原则 · 实战度量	· 价值矩阵 · 实施路径 · 法本保障
P03 — P06	P07 — P09	P10 — P13	P14 — P22	P23 — P26	P27 — P30

CHAPTER 01

核心命题

个人提效 $\neq$ 组织提效

一切问题的起点 —— AI 工具下沉到个人后,组织为什么没变快

个人都在用 AI,版本还是按原节奏发



个人层面:已经变快了

产品经理用 AI 生成原型

工程师用 AI 读码、写码、补测试

测试用 AI 整理用例、分析缺陷

架构师用 AI 做影响分析

每个人似乎都变快了一些



组织层面:感受不到提速

需求仍然在排队

版本仍按原节奏推进

跨团队沟通、联调评审仍反复

测试环境仍然紧张

组织整体并没有同步变快

个人提效 ≠ 组织提效

个人提效已经发生,组织提效没有自然发生

成本结构变了,但协作链路没变

瓶颈从"开发产能"转移到"协作链路",链路不变则组织不变快

1 ⚡ 成本结构变了

代码理解/生成、测试补充、缺陷分析的成本正在下降
"写出代码"从最稀缺的能力,变成可被 AI 大幅放大的能力

2 🎯 瓶颈转移了

新瓶颈在:需求是否清楚 · 架构边界是否合理
测试能否自动验证 · 环境/数据能否快速准备 · 知识能否复用

3 🔗 协作链路没变 → 组织不变快

软件交付是一条协作链:需求 → 评审 → 合入 → 验证 → 发布 → 运营
只给每人加 AI 工具,链路不变 → 协作链路仍是瓶颈 → 组织不变快

软件交付协作链

需求

方案

编码

测试

发布

运营

协作链不变
个人变快被链路吃掉
→ 必须重构生产关系

先建驾驭框架,把个人变快转化为组织系统提速

不是再卖一个 AI 工具,而是让 AI 在组织里可控地跑起来



让"个人用 AI 变快",变成"组织级系统提速"

CHAPTER 02

产品全景

FarAI 研发流水线

一个平台,四层架构,把人、智能体、资产、数据、治理协同起来

四层架构 —— 从底座到治理的人智协同平台

FarAI 以智能系统为中枢,让研发组织的人、智能体、资产、数据和治理协同运转



④ 治理与度量层

风险分级 · Human-in-Loop · AI FinOps · 效能度量

先建安全网,再放大自主度



③ 人智协同平台 (Delivery Workbench)

桌面智能体 · 云端智能体 · 任务/权限/审计 · Skill 治理

连接人、任务、代码、智能体



② Harness 驾驭框架(工程底座)

CLI 化 · fullcheck · CLI E2E · hooks · traceld · Agent OS

Agent 可控的前提



① 知识与本体底座

本体网络 · 语义连接 · Agent 记忆 · 经验进化

让知识可被人和 Agent 共同理解

向上
承载场景

向下
沉淀能力

四大场景串联成完整研发闭环,四层架构逐层支撑

四个场景不是孤立的点,而是覆盖需求 → 交付 → 知识 → 治理的研发全链路



CHAPTER 03

组织重塑

新角色 · 新组织 · 新流程

用新的生产关系承接新的生产力 —— 个人提效转化为组织系统提效的组织载体

Developer → Builder

FDE 三层协同

内部开源式流动

新角色 —— 从 Developer 到 Builder,Builder 负责交付闭环

人不再只负责某个狭窄环节,而是和 AI、Agent、平台、资产一起把业务问题构建成为可运行的系统



 **Builder 化 ≠ 专家消失**

Builder 交付闭环 · Expert 能力沉淀

架构/性能/安全/UX/前端/后端/测试/数据 Expert 仍在,从"审批关口"变成"能力中心",负责定义标准、沉淀资产、建设工具链和质量门禁

新组织 —— FDE 团队、产品团队、平台团队三层协同

三层不是上下级审批关系,而是基于资产和智能体协同的能力流动关系



新流程 —— 代码和能力在三层间流动,内部开源机制

流程的关键不是"谁先批准",而是"谁能快速贡献,谁来持续熵减,谁负责稳定复用"



FDE 团队

快速探索 · 直接改产品代码,必要时改平台代码

围绕四大闭环快速推进,产品/平台智能体参与评审但非低风险阻塞点



代码贡献 + 场景经验



产品团队

产品化沉淀 · 复审、重构、熵减、测试强化、产品化发布

前线代码不一定一开始最优,但只要验证业务价值,就应被吸收沉淀



平台改动 + 共性需求



平台团队

底座化沉淀 · 复审、重构、优化、强化测试、平台化发布

把共性能力沉淀成稳定底座,再推回产品团队和 FDE 使用



反哺

平台 →
产品 →
FDE



这个流程跑得起来的前提:自动化测试必须足够强

只有测试、环境、数据、质量门禁可自动化验证,产品/平台团队才敢对前线代码重构,FDE 才敢更快探索

→ 这正是第四章 Harness 驾驭框架要解决的事

CHAPTER 04

四大场景方案

每个场景一个完整闭环

四个场景,每个解决一个"AI 能切入的高价值环节"

S1 需求闭环

S2 交付闭环

S3 知识底座

S4 治理 FinOps

S1 需求不清反复返工,PRD 进开发前没和真实系统碰撞

⚠️ 痛点

产品闭门写 PRD → 进开发才发现接口不存在 / 数据结构不符 / 边界没想清 → 返工 → 排期延误
需求阶段的返工,是组织提效最大的隐形漏斗

现状流程 vs FarAI 流程

现状



FarAI



📄 输出物

高质量 PRD

边界/异常/状态流已澄清

澄清问题清单

Agent 反向推动

验收标准

可验证的验收点

需求契约

契约驱动开发

S1 让需求在进入开发前,就和真实系统发生碰撞

编码 Agent 反向提问,把需求缺陷消灭在开发前

⚡ FarAI 能力

- 1  **编码 Agent 反读代码库**
读取接口契约、数据结构、历史需求、现有配置
- 2  **自动识别需求缺陷**
歧义、边界缺失、验收标准不全
- 3  **生成澄清问题清单**
反向推动产品澄清边界、异常、权限、状态流
- 4  **输出可验证需求契约**
契约驱动开发,需求即测试基线

↗ 价值

↓ **40-50%**

需求返工率
(预估)

↑ **2-3 倍**

需求一次澄清通过率
(预估)

开发前消缺

隐形成本节省(预估)

S2 AI 写的代码不敢合、不敢重构,质量靠人工把关

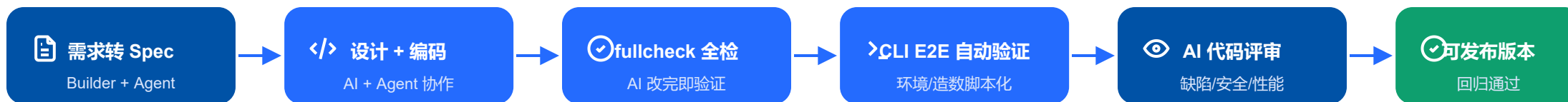
⚠️ 痛点

工程师用 AI 写代码很快 → 但代码评审靠人肉、测试靠人工、环境数据靠手动准备

→ 不敢快速合并、不敢重构、不敢频繁发布

交付闭环:质量内嵌在每一步

质量不是最后一道关,而是交付闭环每一步的内置能力



没有自动化安全网, AI 写得再快组织也不敢合并、重构、发布

Harness 先行 → AI 改完即可验证 → 组织才敢让 AI 高频自主

S2 Harness + 自动化测试,让 AI 改完即可验证可发布

自动化测试安全网越强,AI 能承担的自主执行范围越大

⚙️ FarAI 能力(Harness 组件)

- 1 fullcheck 全量检查**
单测/接口/契约/E2E/回归,AI 改完即验证
- 2 CLI E2E 自动化**
环境/数据/造数脚本化,Agent 可自主验证
- 3 AI 代码评审**
提前发现缺陷、安全、性能、测试缺口
- 4 架构规则 as Code**
分层依赖/契约兼容/越权访问持续守护
- 5 traceld 全链路追踪**
Agent 每一步可追溯、可复盘

↗️ 价值

↓ **40-60%**

返工/缺陷率
(预估)

↑ **3-5 倍**

安全合并/重构吞吐
(预估)

AI 自主执行范围

随安全网强度放大(预估)

S3 Agent 拿不到准确上下文,老找错代码、误解业务

⚠️ 痛点:知识散落,Agent 只能"猜"

需求在 Confluence · 设计在 Figma · 代码在 Git · 测试在 TestRail · 契约在 Swagger · 规则在文档

→ Agent 检索只能靠关键词/向量相似度

→ 找错上下文、误解概念、遗漏约束 → 生成看似合理实则错误的方案

三层资产结构:不是堆文件,是用本体网络把资产连起来



第三层 · 能力资产

Agent 记忆 · Skill · Prompt · 工具调用模板 · 评测集 · 专家规则



第二层 · 语义连接层

本体网络 + 知识图谱:概念 · 系统 · 数据 · 规则 · 接口 · 测试 · 经验 全语义关联



第一层 · 底层工程资产

需求 · 设计 · 代码 · 测试 · 契约/API · 运行日志 · 评测集 · Agent 执行轨迹

Agent 不再靠关键词"猜",而是沿语义关系精确检索

S3 本体网络让 Agent 沿业务语义和工程依赖,找到准确上下文

Agent 不再“猜”,而是沿语义关系精确检索

FarAI 能力

-  **本体网络**
业务概念/系统边界/数据实体/接口契约/规则语义关联
-  **知识联通底座**
本体网络 + 语义检索 + 向量检索 + 代码引用关系
-  **Agent 记忆**
任务/工具/文档/失败/纠偏过程数据结构化保存
-  **经验进化底座**
Prompt/Skill/规则/最佳实践 版本化、评测、持续改进

价值

分钟级 → 秒级

上下文获取时效
(预估)

↑50%+

知识/资产复用率
(预估)

Agent 越用越准

组织经验持续进化成能力(预估)

S4 AI 效能说不清,Token 钱花得没底;管太死又把速度抵消

⚠️ 痛点:治理两难 + 效能黑箱

两难:不管 → 怕 AI 乱跑出事(质量/安全/合规);管太死 → 每步人工审批,速度优势被抵消

黑箱:Token 成本/模型调用成本/Agent 执行成本没有度量

→ "用了多少 AI"清楚,"带来什么收益"说不清

治理五步阶梯:先建安全网,再放大自主度

不是让 AI 慢下来,而是让 AI 在可控边界内跑得更快



不是堆审批,而是先建安全网,再放大自主度

S4 先建安全网,再放大自主度,让速度质量责任价值同时可控

风险分级决定自主度,效能度量决定投入产出

FarAI 能力

-  **风险分级行权**
低风险 Agent 自主 · 中风险确认 · 高风险(资金/权限/隐私)人工把关
-  **AI FinOps**
Token/Agent 执行成本/成功率/返工率/缺陷率全量度量
-  **效能度量**
交付周期/吞吐/测试覆盖/上线后价值,工具使用量 → 真实影响
-  **能力差异洞察**
同样工具不同用法,定位培训/Skill 复用/团队建设重点

价值

AI 投入产出可度量

从"用了多少"到"带来什么"(预估)

↓ **30-50%**
高风险变更漏审风险
(预估)

Token ROI 可视化

成本-收益持续追踪(预估)

CHAPTER 05

工程底座

Harness 驾驭框架

Harness 是 Agent 可控的前提 —— 给技术负责人的工程视图

Harness 9 大组件 —— 驾驭 Agent 的工程抓手

9 个工程组件,把"Agent 能跑"升级为"Agent 可控可验证可追溯"

01 > CLI Harness

CLI 化是基础
让 Agent 能调用代码,代码
具备可被 Agent 操作的接口

02 ✓ fullcheck 全量检查

AI 改完即验证
单测/接口/契约/E2E/回归
全量自动检查,反馈闭环

03 端 CLI E2E 端到端

端到端 CLI 自动化
环境准备/数据构造/造数
全流程脚本化、自动化

04 S Agent traceId

执行全链路可追踪
每个任务、每步操作
可追溯、可复盘、可审计

05 Agent OS

Agent 操作系统
身份/权限/任务队列
Agent 运行的系统底座

06 UI Harness

UI 自动化驾驭
界面层 Agent 可操作
UI 验证脚本化、可重放

07 API Harness

API 自动化驾驭
接口契约可验证
契约兼容性持续守护

08 SOP Harness

标准作业流程驾驭
高频任务沉淀成 SOP
流程可复用、可治理

09 Harness flake

框架自身稳定性治理
驾驭框架本身的稳定性
抖动检测、修复、加固

先建 Harness 再放 Agent,顺序错了全是反模式

Harness 先行,Agent 在 Harness 内跑;先放 Agent 再补 Harness = 不可控

✓ 正确顺序

Harness First

01

Harness 先建
CLI / fullcheck / E2E / hooks 工程底座就位

02

Agent 在 Harness 内跑
Loop / Graph / 任务编排 在驾驭框架内运行

03

feedback 反哺 Harness
Agent 执行反馈持续强化驾驭框架

可控 → 可验证 → 可持续放大自主度

✗ 反模式

Agent First

Loop Engineering
先让 Agent 跑循环,再补约束
→ 循环失控,事故已发生,补不回来

Graph Engineering
先让 Agent 跑任务图,再补治理
→ 任务图失控,治理滞后于破坏

共性:顺序错了
后期补 Harness 成本极高,且事故已发生

先建安全网,再放大自主度

工程底座建成后的度量参考 —— 规模化 AI 协同的真实形态

Harness 先行不是理论,以下是工程化 AI 协同的可观察度量参考



175 万+

单库规模(行)

Agent 可协同的代码库
量级参考



34%+

代码 CLI 化率

Agent 可调用的代码
占比参考



35/天+

Agent 协同提交

日均提交强度
人智协同密度参考



9 大

Harness 组件

驾驭框架的工程抓手
CLI/fullcheck/E2E 等

Harness 先行的真实形态:规模化、可验证、高频协同

百万行级代码库 · 三成以上 CLI 化 · 日均数十次 Agent 协同提交

这才是"个人提效转化为组织级系统提效"的工程化形态

CHAPTER 06

价值、实施与保障

从试点到规模化的路径

不要先改组织,先让一个产品跑起来

四维价值 —— 效率 · 质量 · 成本 · 进化

FarAI 研发流水线从四个维度,把个人提效升级为组织系统提效

⚡ 效率

速度类指标

↓ **40-50%**

需求返工率(预估)

↑ **3-5 倍**

安全合并吞吐(预估)

个人变快 → 小队/产品线/组织级系统提速

🛡️ 质量

风险类指标

↓ **40-60%**

缺陷/返工率(预估)

↓ **30-50%**

高风险漏审(预估)

Harness 安全网让 AI 改完即可验证

📁 成本

资源类指标

分钟级 → 秒级

上下文获取时效(预估)

Token ROI

持续可视化(预估)

AI FinOps 让"用了多少"升级为"带来什么收益"

🔄 进化

长期复利

↑ **50%+**

知识/资产复用率(预估)

越用越准

组织经验成能力(预估)

本体网络 + Agent 记忆 + 经验进化底座持续复利

六阶段落地 —— 先让一个产品具备 AI 可协同的工程底座

不要先大规模改组织,先跑通一个试点,再复制到更多产品和领域



每阶段沉淀 5+2 范式的一部分;试点有效再复制

不要先大规模改组织,先让一个产品具备 AI 可协同的工程底座,再复制到更多产品和领域

为什么是法本 —— 四重保障托底 FarAI 研发流水线落地

能力底座 + 交付经验 + 安全合规 + 生态合作,托底端到端落地

能力底座	交付经验	安全合规	生态合作
FarAI 全栈产品矩阵 自研 Agent / Skill 框架 本体网络与知识工程 Harness 工程方法论 从模型、Agent、Skill、知识到底座的全栈能力 技术产品全栈自研	多行业研发组织落地 标准化实施方法论 试点-复制路径成熟 多产品线规模化实践 金融/制造/科技等行业研发效能落地经验 方法论 + 标杆案例	企业级数据安全 权限与审计 风险分级行权 私有化部署支持 数据不出域、权限可控、操作可审计 企业级·国产化	主流大模型集成 编码工具打通 DevOps 工具链接入 行业标杆客户协同 开放生态、持续演进的技术合作 开放·共建



让 AI 在你的研发组织里, 可控地跑起来

FarAI 研发流水线 · 个人提效 → 组织提效

 扫码咨询

 法本信息 Farben  售前对接